SEIR Epidemic Model

A Compartmental Model of Infectious Disease Transmission

Language: Python
Application Domain: Mathematical Epidemiology
Date: March 2026

1. Introduction

The SEIR model is a classical compartmental framework in mathematical epidemiology used to simulate the temporal dynamics of infectious disease propagation within a closed population. The acronym SEIR denotes the four mutually exclusive population states: Susceptible (S), Exposed (E), Infectious (I), and Recovered (R). Unlike the simpler SIR model, the SEIR formulation explicitly accounts for the latency period — the interval between initial infection and the onset of infectiousness — rendering it more realistic for diseases with a non-negligible incubation window, such as COVID-19, influenza, and Ebola.

The Python implementation documented herein encapsulates the SEIR dynamics within an object-oriented class structure. The model is solved by numerical integration via discrete time-step iteration, and the results are visualised using the Matplotlib library. This document provides a comprehensive technical breakdown of each constituent component: its purpose, mathematical basis, input parameters, and interaction with the broader system.

2. Mathematical Foundation

2.1 Compartmental Structure

The total population N is partitioned at every time step t into four non-overlapping compartments satisfying the conservation identity:

$$\mathrm{S(t) + E(t) + I(t) + R(t) = N,\ \forall t}$$

In this implementation, the population is normalised to N = 1, so all compartment values represent fractions of the total population. This normalisation simplifies the equations and makes results directly interpretable as proportions.

2.2 Discrete-Time Difference Equations

The continuous SEIR ordinary differential equations are approximated by the following first-order forward Euler scheme, advancing in discrete unit time steps:

```
S[t+1] = S[t] - r1·b1·S[t]·I[t] / N  -  r2·b2·S[t]·E[t] / N
E[t+1] = E[t] + r1·b1·S[t]·I[t] / N  +  r2·b2·S[t]·E[t] / N  -  a·E[t]
I[t+1] = I[t] + a·E[t]  -  g·I[t]
R[t+1] = R[t] + g·I[t]
```

Each equation models the rate of flow between compartments per unit time. The bidirectional transmission terms (from both I and E compartments to S) reflect the assumption that individuals in the latent state may also be partially infectious.

3. Model Parameters

3.1 Parameter Definitions

The following table enumerates all parameters accepted by the SEIR class constructor, their symbols, biological interpretations, and the values used in the reference instantiation provided in the __main__ block.

Symbol	Parameter Name	Biological Meaning	Reference Value
`N`	Total population size (normalised)	Fraction denominator; N=1 implies proportional dynamics	1
`E`	Initial exposed fraction	Proportion of population in latent state at t=0	0
`I`	Initial infectious fraction	Proportion of population actively infectious at t=0	0.0001
`R`	Initial recovered fraction	Proportion immune or removed at t=0	0
`r1`	Contact rate modifier 1	Scales transmission rate from I compartment	1
`r2`	Contact rate modifier 2	Scales transmission rate from E compartment	1
`b1`	Transmission probability (I → S)	Per-contact infection probability from infectious individuals	0.148

Symbol	Parameter Name	Biological Meaning	Reference Value
`b2`	Transmission probability (E → S)	Per-contact infection probability from exposed individuals	0.148
`a`	Incubation rate (α)	Rate of progression E→I; reciprocal 1/a gives the mean latency duration in the chosen time unit	0.13
`g`	Recovery rate (γ)	Rate of progression I→R; reciprocal 1/g gives the mean infectious duration in the chosen time unit	0.06
`T`	Time vector	List of discrete time indices for simulation	range(0,150)

3.2 Derived Epidemiological Quantities

Several standard epidemiological metrics can be derived directly from the model parameters:

Quantity	Formula	Value (Reference Run)
Basic Reproduction Number (R_0)	`(r1·b1 + r2·b2) / g`	$R_0 \approx 4.93$
Mean Latency Duration	`1 / a`	1 / 0.13 ≈ 7.69 time steps
Mean Infectious Duration	`1 / g`	1 / 0.06 ≈ 16.67 time steps
Attack Rate (estimated)	$S(\infty) = 1 - R(\infty)$	Determined numerically

Note: An R_0 value greater than 1 indicates a self-sustaining epidemic. The numerical value of R_0 depends entirely on the chosen parameters and does not imply any specific pathogen; its interpretation is meaningful only within the context of the calibration or scenario under study.

4. Class Architecture

4.1 Class Overview

The entire model is encapsulated in a single Python class named `SEIR`. This object-oriented design promotes modularity, reusability, and separation of concerns. The class exposes three distinct interfaces: the constructor for parameter initialisation, a computational method for numerical integration, and a visualisation method for result rendering.

```
class SEIR:
    def __init__(self, T, N, E, I, R, r1, r2, b1, b2, a, g)  # Initialise model state
    def calc(self)                                            # Execute numerical
integration
    def plot(self)                                            # Render time-series
visualisation
```

4.2 Constructor: __init__

The constructor method initialises the model state by accepting all epidemiological parameters as arguments. Compartment histories are stored as Python lists — each initialised with a single seed value — to which subsequent time steps are appended during the `calc()` phase. This list-based accumulation pattern enables memory-efficient incremental computation.

```
def __init__(self, T, N, E, I, R, r1, r2, b1, b2, a, g):
    self.N  = N          # Total (normalised) population
    self.E  = [E]        # Exposed history  (seed: t=0)
    self.I  = [I]        # Infectious history (seed: t=0)
    self.R  = [R]        # Recovered history (seed: t=0)
    self.S  = [N - I]    # Susceptible = N minus initial infectious
    self.r1 = r1         # Contact modifier for I compartment
    self.r2 = r2         # Contact modifier for E compartment
    self.b1 = b1         # Transmission probability from I
    self.b2 = b2         # Transmission probability from E
    self.a  = a          # Latency-to-infectious transition rate
    self.g  = g          # Recovery rate
    self.T  = T          # Time vector (list of discrete time indices)
```

Key design decision: The initial susceptible count is computed as `N - I` rather than `N - E - I - R`. Since the reference run sets E = R = 0, this is algebraically equivalent; however, in general initialisations with non-zero E or R, this would introduce an inconsistency.

4.3 Computational Method: calc()

The `calc()` method performs the core numerical integration by iterating through the time vector and appending updated compartment values at each step using the forward Euler approximation of the SEIR differential equations.

```
def calc(self):
    # Guard clause: skip if already computed
    if len(self.T) == len(self.S):
        return

    for i in range(0, len(self.T) - 1):
        # New susceptible: reduce by infections from I and E
        self.S.append(self.S[i]
            - self.r1 * self.b1 * self.S[i] * self.I[i] / self.N
            - self.r2 * self.b2 * self.S[i] * self.E[i] / self.N)

        # New exposed: gain from infections, lose to progression
        self.E.append(self.E[i]
            + self.r1 * self.b1 * self.S[i] * self.I[i] / self.N
            + self.r2 * self.b2 * self.S[i] * self.E[i] / self.N
            - self.a * self.E[i])

        # New infectious: gain from E progression, lose to recovery
        self.I.append(self.I[i] + self.a * self.E[i] - self.g * self.I[i])

        # New recovered: accumulate recoveries from I
        self.R.append(self.R[i] + self.g * self.I[i])
```

Guard clause: The opening conditional `if len(self.T) == len(self.S)` prevents redundant computation if `calc()` is invoked multiple times. This is a lightweight idempotency guard that ensures the simulation executes exactly once.

4.4 Visualisation Method: plot()

The `plot()` method renders a time-series line graph of all four compartments using Matplotlib. It also lazily invokes `calc()` if the simulation has not yet been executed, enabling the caller to produce a plot with a single method call.

```
def plot(self):
    import matplotlib.pyplot as plt
    import matplotlib as mpl
    mpl.rcParams['axes.unicode_minus'] = False  # Fix minus sign rendering

    if len(self.S) == 1:   # Lazy evaluation: compute only if needed
        self.calc()

    plt.figure()
    plt.plot(self.T, self.S, color='b', label='Susceptible')
    plt.plot(self.T, self.E, color='y', label='Exposed')
    plt.plot(self.T, self.I, color='r', label='Infectious')
    plt.plot(self.T, self.R, color='g', label='Recovered')
    plt.legend()
    plt.xlabel('Time')
    plt.ylabel('Proportion of population')
    plt.show()
```

5. Execution and Reference Run

5.1 Entry Point

The `if __name__ == '__main__'` guard ensures that the reference simulation only executes when the script is run directly, and not when imported as a module. This is standard Python practice for scripts that serve dual roles as importable libraries and standalone executables.

```
if __name__ == '__main__':
    T = [i for i in range(150)]           # 150 discrete time steps
    s = SEIR(T, 1, 0, 0.0001, 0,          # N, E0, I0, R0
             1, 1,                        # r1, r2
             0.148, 0.148,                # b1, b2
             0.13, 0.06)                  # alpha, gamma
    s.plot()
```

5.2 Expected Simulation Output

For the reference parameters, the simulation produces the following qualitative behaviour over 150 time steps:

Compartment	Colour	Expected Trajectory
Susceptible (S)	Blue	Monotonically decreasing sigmoid curve; approaches asymptote as herd immunity threshold is reached
Exposed (E)	Yellow	Unimodal curve; rises as epidemic spreads, peaks roughly at epidemic midpoint, then declines
Infectious (I)	Red	Unimodal curve peaking after E; represents the clinical epidemic curve visible to health systems
Recovered (R)	Green	Monotonically increasing sigmoid; mirrors the depletion of S, approaching $1 - S(\infty)$

6. Summary

The SEIR class provides a concise, self-contained implementation of a compartmental epidemic model with bidirectional transmission from both the Exposed and Infectious populations. The three-method interface (constructor, calc, plot) embodies a clean separation of concerns: state initialisation, numerical computation, and result presentation are each isolated into independent, testable units. The idempotency guard in calc() and the lazy evaluation pattern in plot() collectively prevent redundant computation without requiring the caller to manage execution state explicitly.

For applications requiring higher fidelity — such as policy scenario modelling or calibration against empirical case data — extensions should consider adopting continuous ODE integration, stochastic formulations, age-structured contact matrices, and time-varying parameters.

References

[1] Kermack, W. O., & McKendrick, A. G. (1927). A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772), 700–721.

[2] Hethcote, H. W. (2000). The mathematics of infectious diseases. *SIAM Review*, 42(4), 599–653.

[3] Anderson, R. M., & May, R. M. (1991). *Infectious Diseases of Humans: Dynamics and Control*. Oxford University Press.

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.